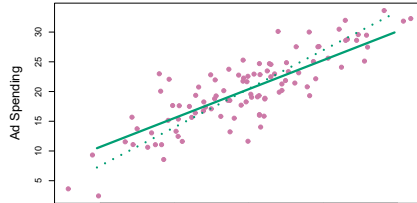


6.3.2 Partial Least Squares

PCR approach vừa mô tả xác định các linear combination, hay direction, biểu diễn tốt nhất các predictor $X_1, \dots, X_p$. Các direction này được xác định theo cách *unsupervised*, vì response Y không được dùng để hỗ trợ xác định principal component directions. Nói cách khác, response không supervise việc nhận diện principal components. Vì vậy PCR có một nhược điểm: không có gì bảo đảm những direction giải thích predictors tốt nhất cũng là những direction tốt nhất để predict response.



HÌNH 6.21. Với advertising data, first PLS direction (solid line) và first PCR direction (dotted line) được hiển thị.

Các unsupervised method được bàn thêm trong Chapter 12. Bây giờ ta giới thiệu *partial least squares* (PLS), một supervised alternative cho PCR. Giống PCR, PLS là dimension reduction method: trước hết xác định một tập feature mới $Z_1, \dots, Z_M$ là các linear combination của original features, rồi fit một linear model bằng least squares sử dụng M feature mới này. Nhưng khác PCR, PLS xác định các feature mới theo cách supervised—tức dùng response Y để tìm những feature vừa approximate original features tốt vừa liên quan tới response. Nói nôm na, PLS cố tìm các direction giúp explain cả response lẫn predictors.

Ta mô tả cách tính first PLS direction. Sau khi standardize p predictors, PLS tạo direction đầu tiên Z_1 bằng cách đặt mỗi ϕ_{j1} trong (6.16) bằng coefficient từ simple linear regression của Y theo X_j . Có thể chứng minh coefficient này tỷ lệ với correlation giữa Y và X_j . Vì vậy khi tính $Z_1 = \sum_{j=1}^p \phi_{j1} X_j$, PLS đặt weight lớn nhất lên các variable có relationship mạnh nhất với response.

Figure 6.21 minh họa PLS trên synthetic dataset, trong đó response là **Sales** ở 100 region và hai predictors là **Population Size** và **Advertising Spending**. Solid green line là first PLS direction, dotted line là first principal component direction. So với PCA, PLS chọn một direction có mức thay đổi theo dimension **ad** nhỏ hơn trên mỗi unit thay đổi theo dimension **pop**. Điều này gợi ý **pop** correlated với response mạnh hơn **ad**. PLS direction không fit predictors sát bằng PCA, nhưng explain response tốt hơn.

Để xác định second PLS direction, trước hết ta adjust từng variable theo Z_1 bằng cách regress từng variable lên Z_1 rồi lấy residual. Các residual này có thể được xem là phần information còn lại chưa được first PLS direction explain. Sau đó ta tính Z_2 từ orthogonalized data này theo đúng cách đã dùng để tính Z_1 từ original data.

Iterative approach này có thể lặp M lần để xác định nhiều PLS components $Z_1, \dots, Z_M$. Cuối cùng, ta dùng least squares để fit linear model predict Y bằng $Z_1, \dots, Z_M$, hoàn toàn tương tự PCR.

Giống PCR, số M partial least squares directions dùng trong PLS là một tuning parameter thường được chọn bằng cross-validation. Ta thường standardize cả predictors lẫn response trước khi thực hiện PLS.

PLS phổ biến trong chemometrics, nơi nhiều variable xuất phát từ digitized spectrometry signals. Trong thực tế nó thường không perform tốt hơn ridge regression hoặc PCR. Supervised dimension reduction của PLS có thể giảm bias, nhưng cũng có khả năng tăng variance, nên overall benefit của PLS so với PCR có thể gần như triệt tiêu.

6.4 Considerations in High Dimensions

6.4.1 High-Dimensional Data

Phần lớn statistical technique truyền thống cho regression và classification được thiết kế cho *low-dimensional setting*, trong đó n , số observations, lớn hơn nhiều so với p , số features. Một phần nguyên nhân là trong phần lớn lịch sử của statistics, đa số scientific problem cần statistics đều low-dimensional. Ví dụ, để xây model predict blood pressure của patient dựa trên age, sex và body mass index (BMI), ta có ba predictors, hay bốn nếu tính cả intercept, và có thể vài nghìn patient với đủ measurements. Khi đó $n \gg p$, nên problem là low-dimensional. (Ở đây *dimension* nói tới độ lớn của p .)

low-dimensional

Trong khoảng 20 năm qua, technology mới đã thay đổi cách data được collect trong finance, marketing, medicine và nhiều field khác. Hiện nay việc collect gần như unlimited number of feature measurements (p rất lớn) là bình thường. Trong khi p có thể cực lớn, số observations n thường bị giới hạn bởi cost, sample availability hoặc các consideration khác. Hai example:

1. Thay vì predict blood pressure chỉ từ age, sex và BMI, có thể thu thêm measurement cho nửa triệu single nucleotide polymorphisms (SNPs; các DNA mutation riêng lẻ tương đối phổ biến trong population) để đưa vào predictive model. Khi đó $n \approx 200$ và $p \approx 500,000$.
2. Một marketing analyst muốn hiểu online shopping pattern có thể coi toàn bộ search term người dùng nhập vào search engine là features. Cách này đôi khi gọi là “bag-of-words” model. Researcher đó có thể chỉ có search history của vài trăm hoặc vài nghìn user đồng ý chia sẻ information. Với mỗi user, mỗi trong p search term được score present (0) hoặc absent (1), tạo nên một binary feature vector rất lớn. Khi đó $n \approx 1,000$ còn p lớn hơn nhiều.

Dataset có nhiều features hơn observations thường được gọi là *high-dimensional*. Classical approach như least squares linear regression không phù hợp trong setting này. Nhiều issue của high-dimensional data đã xuất hiện trước đây trong sách vì chúng cũng có khi $n > p$, gồm bias-variance trade-off và danger of overfitting. Dù luôn quan trọng, các issue này đặc biệt nghiêm trọng khi số features rất lớn so với số observations.

high-dimensional

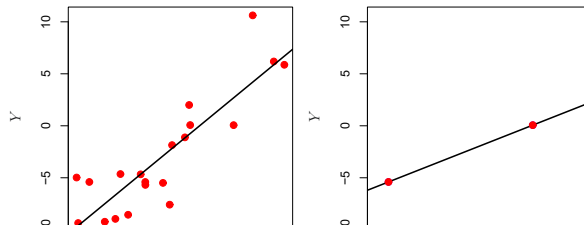
Ta định nghĩa high-dimensional setting là case số features p lớn hơn số observations n . Tuy nhiên, các consideration sắp bàn cũng áp dụng khi p chỉ hơi nhỏ hơn n , và tốt nhất luôn ghi nhớ chúng khi thực hiện supervised learning.

6.4.2 What Goes Wrong in High Dimensions?

Để minh họa vì sao cần extra care và specialized technique cho regression và classification khi $p > n$, ta xem điều gì xảy ra nếu áp dụng một statistical technique vốn không dành cho high-dimensional setting. Ta dùng least squares regression làm ví dụ, nhưng các concept tương tự áp dụng cho logistic regression, linear discriminant analysis và các classical statistical approach khác.

Khi số features p lớn bằng hoặc lớn hơn số observations n , least squares như Chapter 3 không thể—hay chính xác hơn là không nên—được thực hiện. Lý do rất đơn giản: bất kể features có thật sự relationship với response hay không, least squares sẽ tìm được một set coefficient estimates cho perfect fit trên data, với residual bằng zero.

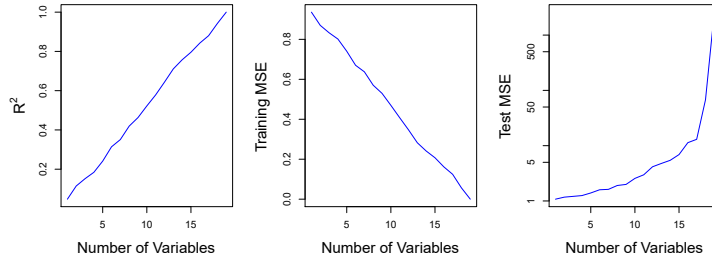
Figure 6.22 minh họa với $p = 1$ feature (cộng intercept) trong hai case: có 20 observations và chỉ có hai observations. Khi có 20 observations, $n > p$ và least squares regression line không fit data hoàn hảo; thay vào đó nó approximate 20 observations tốt nhất có thể. Khi chỉ có hai observations, bất kể value của chúng là gì, regression line sẽ fit data chính xác. Điều này problematic vì perfect fit gần như chắc chắn dẫn tới overfitting. Dù có thể fit training data hoàn hảo trong high-dimensional setting, linear model thu được sẽ thường perform cực kém trên independent test set và vì thế không phải useful model. Thật vậy, line từ right panel Figure 6.22 sẽ perform rất kém nếu test set gồm observations ở left panel. Vấn đề là khi $p > n$ hoặc $p \approx n$, ordinary least squares quá flexible và overfit data.



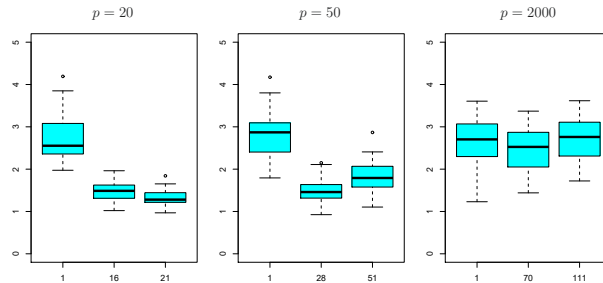
HÌNH 6.22. Left: Least squares regression trong low-dimensional setting. Right: Least squares regression với $n = 2$ observations và hai parameter cần estimate (một intercept và một coefficient).

Figure 6.23 tiếp tục minh họa risk của việc bất cẩn dùng least squares khi p lớn. Data được simulate với $n = 20$ observations, và regression dùng từ 1 đến 20 features, tất cả đều hoàn toàn unrelated với response. Như figure cho thấy, model R^2 tăng lên 1 và training-set MSE tương ứng giảm về 0 khi số features tăng, dù các feature hoàn toàn không liên quan response. Ngược lại, MSE trên independent test set tăng cực mạnh khi thêm features vì additional predictors làm variance của coefficient estimates tăng rất lớn. Nhìn test-set MSE, model tốt nhất chỉ chứa nhiều nhất vài variables. Tuy nhiên, nếu chỉ nhìn R^2 hoặc training-set MSE, ta có thể kết luận sai rằng model có nhiều variables nhất là tốt nhất. Điều này cho thấy cần extra care khi analyze dataset với nhiều variables và phải luôn evaluate model performance trên independent test set.

Trong Section 6.1.3 ta đã thấy nhiều approach adjust training-set RSS



HÌNH 6.23. Trong simulated example với $n = 20$ training observations, các feature hoàn toàn unrelated với outcome lần lượt được thêm vào model. Left: R^2 tăng lên 1 khi thêm nhiều feature. Center: Training-set MSE giảm về 0. Right: Test-set MSE tăng khi thêm feature.



HÌNH 6.24. Lasso được thực hiện với $n = 100$ observations và ba value của p , số features. Trong p features có 20 feature thật sự associated với response. Boxplot cho test MSE với ba value khác nhau của tuning parameter λ trong (6.7). Để dễ interpretation, thay vì report λ , figure report degrees of freedom; với lasso đây chính là số estimated non-zero coefficients. Khi $p = 20$, test MSE thấp nhất đạt được với mức regularization nhỏ nhất. Khi $p = 50$, test MSE thấp nhất đạt được khi regularization đáng kể. Khi $p = 2,000$, lasso perform kém bất kể regularization, do chỉ 20 trong 2,000 features thật sự associated với outcome.

hoặc R^2 để account cho số variables trong least squares model. Đáng tiếc, C_p , AIC và BIC không phù hợp trong high-dimensional setting vì estimate $\hat{\sigma}^2$ trở nên problematic. Chẳng hạn formula $\hat{\sigma}^2$ từ Chapter 3 cho estimate $\hat{\sigma}^2 = 0$ trong setting này. Tương tự, adjusted R^2 cũng gặp vấn đề vì ta có thể dễ dàng đạt adjusted $R^2 = 1$. Rõ ràng cần alternative approach phù hợp hơn với high-dimensional setting.

6.4.3 Regression in High Dimensions

Nhiều method trong chapter này nhằm fit least-squares model ít flexible hơn—như forward stepwise selection, ridge regression, lasso và principal components regression—đặc biệt hữu ích cho regression trong high-dimensional setting. Về bản chất, các approach này tránh overfitting bằng một fitting approach ít flexible hơn least squares.

Figure 6.24 minh họa performance của lasso trong một simulated example đơn giản. Có $p = 20, 50$ hoặc 2,000 features, trong đó 20 feature thật sự associated với outcome. Lasso được fit trên $n = 100$ training observations và mean squared error được evaluate trên independent test set. Khi số features tăng, test-set error tăng. Khi $p = 20$, lowest validation-set error đạt được khi λ trong (6.7) nhỏ; khi p lớn hơn, lowest validation-set error đạt được với λ lớn hơn.

Trong mỗi boxplot, thay vì report λ , degrees of freedom của resulting lasso solution được hiển thị; đây đơn giản là số non-zero coefficient estimates và là một measure của flexibility của lasso fit. Figure 6.24 làm nổi bật ba point quan trọng: (1) regularization hay shrinkage đóng vai trò then chốt trong high-dimensional problem, (2) appropriate tuning-parameter selection là crucial để predictive performance tốt, và (3) test error có xu hướng tăng khi dimensionality của problem (tức số features hoặc predictors) tăng, trừ khi additional features thật sự associated với response.

Point thứ ba là một key principle trong analysis của high-dimensional data, gọi là *curse of dimensionality*. Ta có thể nghĩ rằng tăng số features dùng để fit model sẽ làm quality của model tăng. Nhưng so sánh left và right panels Figure 6.24 cho thấy không nhất thiết vậy: trong example này test-set MSE gần gấp đôi khi p tăng từ 20 lên 2,000. Nói chung, thêm signal features thật sự associated với response sẽ cải thiện fitted model bằng cách giảm test-set error. Ngược lại, thêm noise features không thật sự associated với response làm model tệ đi và tăng test-set error.

Lý do là noise features tăng dimensionality của problem, làm risk of overfitting nghiêm trọng hơn—do noise feature có thể nhận non-zero coefficient chỉ vì chance association với response trên training set—mà không có upside về test-set error. Vì vậy technology mới cho phép collect measurements cho hàng nghìn hay hàng triệu features là con dao hai lưỡi: chúng có thể tạo predictive model tốt hơn nếu features thật sự relevant, nhưng sẽ làm result xấu đi nếu features không relevant. Ngay cả khi relevant, variance phát sinh khi fit coefficient của chúng có thể lớn hơn reduction in bias mà chúng đem lại.

*curse of
dimensionality*

6.4.4 Interpreting Results in High Dimensions

Khi thực hiện lasso, ridge regression hoặc regression procedure khác trong high-dimensional setting, phải rất thận trọng khi report result. Chapter 3 đã nói về multicollinearity, tức các variable trong regression có thể correlated với nhau. Trong high-dimensional setting, multicollinearity trở nên extreme: bất kỳ variable nào trong model cũng có thể viết như linear combination của toàn bộ variables còn lại. Về bản chất, điều này nghĩa là ta không bao giờ biết chính xác variable nào (nếu có) thật sự predictive of outcome, cũng không thể xác định duy nhất best coefficients cho regression. Tối đa, ta chỉ hy vọng assign large regression coefficients cho các variable correlated với những variable thật sự predictive.

Ví dụ, giả sử ta predict blood pressure từ nửa triệu SNPs, và forward stepwise selection cho thấy 17 SNPs tạo good predictive model trên training data. Sẽ sai nếu kết luận 17 SNP này predict blood pressure hiệu quả hơn các SNP không được chọn. Có khả năng tồn tại nhiều tập 17 SNP khác cũng predict tốt tương đương. Nếu obtain independent dataset rồi thực hiện forward stepwise selection, ta có thể nhận một model với tập SNP khác, thậm chí không overlap. Điều đó không làm giảm giá trị của model đã có—chẳng hạn nó có thể predict rất hiệu quả trên independent patient set và hữu ích clinically—nhưng ta không nên overstate result. Cần nói rõ rằng ta chỉ tìm ra một trong nhiều possible model để predict blood pressure và model còn phải được validate trên independent datasets.

Cũng phải đặc biệt cẩn thận khi report error và measure of model fit trong high-dimensional setting. Khi $p > n$, ta dễ dàng obtain một useless model có zero residuals. Vì vậy không bao giờ dùng sum of squared errors, p -values, R^2 statistics hay traditional measure of model fit trên training data làm evidence cho good model fit trong high-dimensional setting. Như Figure 6.23, có thể dễ dàng có $R^2 = 1$ khi $p > n$. Report điều đó có thể khiến người khác hiểu nhầm rằng đã obtain statistically valid, useful model, trong khi nó hoàn toàn không cung cấp evidence như vậy. Thay vào đó, cần report result trên independent test set hoặc cross-validation errors. Ví dụ MSE hoặc R^2 trên independent test set là valid measure of model fit, còn MSE trên training set chắc chắn không phải.

6.5 Lab: Linear Models and Regularization Methods

Trong lab này ta implement nhiều technique đã bàn trong chapter. Trước hết import các library quen thuộc ở top level.

```
In [1]: import numpy as np
import pandas as pd
from matplotlib.pyplot import subplots
from statsmodels.api import OLS
import sklearn.model_selection as skm
import sklearn.linear_model as skl
from sklearn.preprocessing import StandardScaler
from ISLP import load_data
from ISLP.models import ModelSpec as MS
from functools import partial
```

Ta cũng gom các import mới cần cho lab này.

```
In [2]: from sklearn.pipeline import Pipeline
        from sklearn.decomposition import PCA
        from sklearn.cross_decomposition import PLSRegression
        from ISLP.models import \
            (Stepwise,
             sklearn_selected,
             sklearn_selection_path)
        !pip install l0bnb
        from l0bnb import fit_path
```

Package `l0bnb` được install ngay trong lúc chạy. Lưu ý lệnh `!pip install` được escape; đây là một system command riêng.

6.5.1 Subset Selection Methods

Ở đây ta implement các method giảm số parameter trong model bằng cách restrict model vào một subset của input variables.

Forward Selection

Ta áp dụng forward-selection approach lên data `Hitters`. Mục tiêu là predict `Salary` của một baseball player dựa trên các statistic về performance ở previous year.

Trước hết, variable `Salary` bị missing ở một số player. Function `np.isnan()` có thể identify missing observations; nó trả về array cùng shape với input vector, với `True` cho missing element và `False` cho non-missing element. Method `sum()` sau đó đếm tổng số missing element.

`np.isnan()`
`sum()`

```
In [3]: Hitters = load_data('Hitters')
        np.isnan(Hitters['Salary']).sum()
```

Out[3]: 59

Ta thấy `Salary` missing cho 59 player. Method `dropna()` của data frame mặc định remove mọi row có missing value ở bất kỳ variable nào (xem `Hitters.dropna()`).

```
In [4]: Hitters = Hitters.dropna();
        Hitters.shape
```

Out[4]: (263, 20)

Trước tiên ta chọn best model bằng forward selection dựa trên C_p (6.2). Score này không có sẵn như metric trong `sklearn`, vì vậy ta tự define function để compute và dùng nó làm scorer. Mặc định `sklearn` cố maximize score, nên scoring function của ta compute negative C_p statistic.

```
In [5]: def nCp(sigma2, estimator, X, Y):
        "Negative Cp statistic"
        n, p = X.shape
        Yhat = estimator.predict(X)
        RSS = np.sum((Y - Yhat)**2)
        return -(RSS + 2 * p * sigma2) / n
```

Ta cần estimate residual variance σ^2 , là first argument của scoring func-

tion trên. Ta fit largest model, dùng toàn bộ variables, rồi estimate σ^2 từ MSE của model đó.

```
In [6]: design = MS(Hitters.columns.drop('Salary')).fit(Hitters)
Y = np.array(Hitters['Salary'])
X = design.transform(Hitters)
sigma2 = OLS(Y,X).fit().scale
```

Function `sklearn_selected()` chỉ expect scorer có ba arguments—ba argument cuối trong definition `nCp()` ở trên. Ta dùng `partial()` đã gặp ở Section 5.3.3 để freeze first argument bằng estimate σ^2 .

```
In [7]: neg_Cp = partial(nCp, sigma2)
```

Bây giờ có thể dùng `neg_Cp()` làm scorer cho model selection. Ngoài score, ta cần specify search strategy. Việc này được thực hiện qua object `Stepwise()` trong package `ISLP.models`. Method `Stepwise.first_peak()` chạy forward stepwise cho tới khi thêm variable nữa không còn improve evaluation score. Tương tự, `Stepwise.fixed_steps()` chạy một fixed number of steps trong stepwise search.

```
In [8]: strategy = Stepwise.first_peak(design,
                                     direction='forward',
                                     max_terms=len(design.terms))
```

Ta fit linear regression với `Salary` làm outcome bằng forward selection. Function `sklearn_selected()` trong `ISLP.models` nhận một model từ `statsmodels` cùng search strategy, rồi select model qua method `fit`. Nếu không specify scoring argument, score mặc định là MSE, nên cả 19 variables sẽ được chọn (output không hiển thị).

sklearn_selected()

```
In [9]: hitters_MSE = sklearn_selected(OLS,
                                     strategy)
hitters_MSE.fit(Hitters, Y)
hitters_MSE.selected_state_
```

Dùng `neg_Cp` cho model nhỏ hơn, đúng như kỳ vọng, chỉ có 10 variables được chọn.

```
In [10]: hitters_Cp = sklearn_selected(OLS,
                                     strategy,
                                     scoring=neg_Cp)
hitters_Cp.fit(Hitters, Y)
hitters_Cp.selected_state_
```

```
Out[10]: ('Assists',
          'AtBat',
          'CAtBat',
          'CRBI',
          'CRuns',
          'CWalks',
          'Division',
          'Hits',
          'PutOuts',
          'Walks')
```

Choosing Among Models Using the Validation Set Approach and Cross-Validation

Thay cho C_p , ta có thể dùng cross-validation để select model trong forward selection. Ta cần một method lưu full path của các model được tìm trong forward selection và cho phép prediction với từng model. Estimator `sklearn_selection_path()` trong `ISLP.models` làm được việc này. Function `cross_val_predict()` tính cross-validated predictions cho từng model dọc path, từ đó ta evaluate cross-validated MSE trên path.

Ta define một strategy fit toàn bộ forward-selection path. Dù `sklearn_selection_path()` có nhiều parameter choice, ở đây dùng defaults, chọn model tại mỗi step theo mức reduction in RSS lớn nhất.

`sklearn_selection_path()`
`cross_val_predict()`

In [11]:

```
strategy = Stepwise.fixed_steps(design,
                                len(design.terms),
                                direction='forward')
full_path = sklearn_selection_path(OLS, strategy)
```

Ta fit full forward-selection path trên `Hitters` và compute fitted values.

In [12]:

```
full_path.fit(Hitters, Y)
Yhat_in = full_path.predict(Hitters)
Yhat_in.shape
```

Out[12]:

(263, 20)

Kết quả là một array fitted values gồm 20 steps, kể cả fitted mean cho null model. Ta dùng nó để evaluate in-sample MSE. Như dự kiến, in-sample MSE improve ở mỗi step, nên phải dùng validation hoặc cross-validation để chọn số steps. Ta fix y-axis từ 50,000 đến 250,000 để so với cross-validation/validation-set MSE bên dưới, cũng như ridge regression, lasso và principal components regression.

In [13]:

```
mse_fig, ax = subplots(figsize=(8,8))
insample_mse = ((Yhat_in - Y[:,None])**2).mean(0)
n_steps = insample_mse.shape[0]
ax.plot(np.arange(n_steps),
        insample_mse,
        'k', # color black
        label='In-sample')
ax.set_ylabel('MSE',
              fontsize=20)
ax.set_xlabel('# steps of forward stepwise',
              fontsize=20)
ax.set_xticks(np.arange(n_steps)[::2])
ax.legend()
ax.set_ylim([50000,250000]);
```

Lưu ý expression `None` trong `Y[:,None]`. Nó thêm một axis (dimension) vào one-dimensional array `Y`, để array này có thể được recycle khi subtract khỏi two-dimensional `Yhat_in`.

Ta sẵn sàng dùng cross-validation để estimate test error dọc model path. Phải chỉ dùng training observations cho mọi aspect của model-fitting, gồm cả variable selection. Vì vậy model tốt nhất ở mỗi size phải được xác định chỉ bằng training observations trong từng training fold. Đây là point tinh tế nhưng quan trọng: nếu full dataset được dùng để select best subset tại mỗi

step, validation-set error và cross-validation error sẽ không phải accurate estimate của test error.

Ta compute cross-validated predicted values bằng 5-fold cross-validation.

In [14]:

```
K=5
kfold = skm.KFold(K,
                  random_state=0,
                  shuffle=True)
Yhat_cv = skm.cross_val_predict(full_path,
                                Hitters,
                                Y,
                                cv=kfold)

Yhat_cv.shape
```

*skm.KFold()
skm.cross_val_
predict()*

Out[14]: (263, 20)

Prediction matrix `Yhat_cv` có cùng shape với `Yhat_in`; khác biệt là prediction ở mỗi row (tương ứng một sample index) được tạo từ model fit trên training fold không chứa row đó.

Tại mỗi model dọc path, ta compute MSE trong từng cross-validation fold, rồi average để lấy mean MSE và dùng individual values để tạo crude estimate của standard error of the mean.⁹ Vì vậy ta cần biết test indices của từng cross-validation split. Có thể lấy bằng method `split()` của `kfold`. Do đã fix random state, mỗi lần split bất kỳ array nào có cùng số row như `Y`, ta recover cùng training/test indices; bên dưới chỉ ignore training indices.

In [15]:

```
cv_mse = []
for train_idx, test_idx in kfold.split(Y):
    errors = (Yhat_cv[test_idx] - Y[test_idx,None])**2
    cv_mse.append(errors.mean(0)) # column means
cv_mse = np.array(cv_mse).T
cv_mse.shape
```

Out[15]: (20, 5)

Bây giờ thêm cross-validation error estimate vào MSE plot, gồm mean error qua năm folds và estimate standard error of the mean.

In [16]:

```
ax.errorbar(np.arange(n_steps),
            cv_mse.mean(1),
            cv_mse.std(1) / np.sqrt(K),
            label='Cross-validated',
            c='r') # color red
ax.set_ylim([50000,250000])
ax.legend()
mse_fig
```

Để lặp lại bằng validation-set approach, ta chỉ đổi `cv` argument thành một validation set: một random split của data thành test và training. Ta chọn test size 20%, gần bằng size mỗi test set trong 5-fold cross-validation.

skm.ShuffleSplit()

⁹Estimate này crude vì năm error estimate dựa trên các overlapping training set và do đó không independent.

```
In [17]: validation = skm.ShuffleSplit(n_splits=1,
                                     test_size=0.2,
                                     random_state=0)
for train_idx, test_idx in validation.split(Y):
    full_path.fit(Hitters.iloc[train_idx],
                  Y[train_idx])
    Yhat_val = full_path.predict(Hitters.iloc[test_idx])
    errors = (Yhat_val - Y[test_idx, None])**2
    validation_mse = errors.mean(0)
```

Giống case in-sample MSE, validation-set approach không cung cấp standard errors.

```
In [18]: ax.plot(np.arange(n_steps),
                 validation_mse,
                 'b--', # color blue, broken line
                 label='Validation')
ax.set_xticks(np.arange(n_steps)[::2])
ax.set_ylim([50000, 250000])
ax.legend()
mse_fig
```

Best Subset Selection

Forward stepwise là greedy selection procedure: ở mỗi step nó augment current set bằng một additional variable. Bây giờ ta áp dụng best subset selection cho `Hitters`; với mỗi subset size, method search best set of predictors.

Ta dùng package `10bnb` để perform best subset selection. Thay vì constrain subset phải có một fixed size, package tạo path of solutions trong đó subset size đóng vai trò penalty thay vì constraint. Distinction này tinh tế, nhưng khác biệt trở nên quan trọng khi cross-validate.

```
In [19]: D = design.fit_transform(Hitters)
D = D.drop('intercept', axis=1)
X = np.asarray(D)
```

Ở đây first column tương ứng intercept đã được exclude vì `10bnb` sẽ fit intercept riêng. Ta tìm path bằng function `fit_path()`.

```
In [20]: path = fit_path(X,
                        Y,
                        max_nonzeros=X.shape[1])
```

Function `fit_path()` trả về list; mỗi value gồm fitted coefficients dưới key `B`, intercept dưới `B0`, cùng vài attribute khác liên quan particular path algorithm. Các detail đó nằm ngoài scope của sách.

```
In [21]: path[3]
```

```
Out[21]: {'B': array([0.         , 3.254844, 0.         , 0.         , 0.         ,
                    0.         , 0.         , 0.         , 0.         ,
                    0.         , 0.677753, 0.         , 0.         , 0.         ,
                    0.         , 0.         , 0.         , 0.         ]),
          'B0': -38.98216739555494,
          'lambda_0': 0.011416248027450194,
          'M': 0.5829861733382011,
          'Time_exceeded': False}
```

Trong example trên, ở fourth step của path có hai nonzero coefficients trong 'B', tương ứng value 0.114 của penalty parameter `lambda_0`. Có thể dùng sequence of fits này để prediction trên validation set như function của `lambda_0`, hoặc với nhiều work hơn bằng cross-validation.

6.5.2 Ridge Regression and the Lasso

Ta dùng package `sklearn.linear_model` (viết tắt `skl`) để fit ridge và lasso regularized linear model trên `Hitters`. Bắt đầu với model matrix `X` (không intercept) đã compute ở phần best subset regression.

Ridge Regression

Ta dùng function `skl.ElasticNet()` để fit cả ridge và lasso. Để fit path của ridge regression models, dùng `skl.ElasticNet.path()`, có thể fit ridge, lasso và hybrid mixture; ridge regression tương ứng `l1_ratio=0`. Tốt nhất nên standardize columns của `X` nếu variables đo ở units khác nhau. Vì `skl.ElasticNet()` không tự normalize, ta phải xử lý. Do standardize trước, để recover coefficient estimates trên original scale thì phải unstandardize coefficients. Parameter λ trong (6.5) và (6.7) được gọi là **alphas** trong `sklearn`; để nhất quán với chapter, bên dưới dùng tên **lambdas**.¹⁰

In [22]:

```
Xs = X - X.mean(0)[None,:]
X_scale = X.std(0)
Xs = Xs / X_scale[None,:]
lambdas = 10*np.linspace(8, -2, 100) / Y.std()
soln_array = skl.ElasticNet.path(Xs,
                                Y,
                                l1_ratio=0.,
                                alphas=lambdas)[1]

soln_array.shape
```

*skl.ElasticNet()
skl.ElasticNet.
path()*

Out[22]: (19, 100)

Ở đây ta extract array of coefficients tương ứng solutions dọc regularization path. Mặc định `skl.ElasticNet.path` fit path trên automatically selected range of λ , trừ case `l1_ratio=0` tạo ridge regression.¹¹ Vì vậy ta explicit grid từ $\lambda = 10^8$ đến 10^{-2} được scale bởi standard deviation của y , gần như phủ toàn bộ range từ null model chỉ có intercept tới least-squares fit.

Mỗi value λ đi với một vector ridge regression coefficients, truy cập bằng một column của `soln_array`. Trong case này `soln_array` là matrix 19×100 , với 19 rows (mỗi predictor một row) và 100 columns (mỗi λ một column).

Ta transpose matrix này và convert thành data frame để dễ view và plot.

In [23]:

```
soln_path = pd.DataFrame(soln_array.T,
                        columns=D.columns,
                        index=-np.log(lambdas))
soln_path.index.name = 'negative log(lambda)'
soln_path
```

¹⁰Tại thời điểm xuất bản, ridge fits như code chunk [22] phát sinh convergence warning không cần thiết; authors kỳ vọng warning này sẽ biến mất khi package trưởng thành hơn.

¹¹Lý do khá technical: với mọi model trừ ridge, có thể tìm smallest λ sao cho mọi coefficient đều zero. Với ridge, value đó là ∞ .

```
Out[23]:
           AtBat      Hits      HmRun      Runs      ...
negative
log(lambda)
-12.310855    0.000800  0.000889  0.000695  0.000851  ...
-12.078271    0.001010  0.001122  0.000878  0.001074  ...
-11.845686    0.001274  0.001416  0.001107  0.001355  ...
-11.613102    0.001608  0.001787  0.001397  0.001710  ...
-11.380518    0.002029  0.002255  0.001763  0.002158  ...
...          ...          ...          ...          ...  ...

100 rows × 19 columns
```

Ta plot path để thấy coefficients vary theo λ . Để control legend location, trước hết set `legend=False` trong plot method rồi thêm legend bằng `legend()` của `ax`.

```
In [24]:
path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Standardized coefficients', fontsize=20)
ax.legend(loc='upper left');
```

(Ta dùng LaTeX formatting trong horizontal label để format Greek λ .) Ta expect coefficient estimates có ℓ_2 norm nhỏ hơn nhiều khi dùng λ lớn so với λ nhỏ. Nhớ rằng ℓ_2 norm là square root của sum of squared coefficient values. Ta display coefficients ở step 40, nơi $\lambda = 25.535$.

```
In [25]:
beta_hat = soln_path.loc[soln_path.index[39]]
lambdas[39], beta_hat
```

```
Out[25]:
(25.535,
 AtBat      5.433750
 Hits       6.223582
 HmRun      4.585498
 Runs       5.880855
 RBI        6.195921
 Walks      6.277975
 Years      5.299767
 ...)
```

Ta compute ℓ_2 norm của standardized coefficients.

```
In [26]:
np.linalg.norm(beta_hat)
```

```
Out[26]: 24.17
```

Ngược lại, dưới đây là ℓ_2 norm khi $\lambda = 2.44e-01$. Coefficient norm lớn hơn nhiều với value λ nhỏ hơn này.

```
In [27]:
beta_hat = soln_path.loc[soln_path.index[59]]
lambdas[59], np.linalg.norm(beta_hat)
```

```
Out[27]: (0.2437, 160.4237)
```

Ở trên ta normalize `X` trước rồi fit ridge model bằng `Xs`. Object `Pipeline()` trong `sklearn` cung cấp cách rõ ràng để tách feature normalization khỏi việc fit ridge model.

```
In [28]: ridge = skl.ElasticNet(alpha=lambdas[59], l1_ratio=0)
        scaler = StandardScaler(with_mean=True, with_std=True)
        pipe = Pipeline(steps=[('scaler', scaler), ('ridge', ridge)])
        pipe.fit(X, Y)
```

Ta cho thấy nó cho cùng ℓ_2 norm như previous fit trên standardized data.

```
In [29]: np.linalg.norm(ridge.coef_)
```

```
Out[29]: 160.4237
```

Lưu ý operation `pipe.fit(X, Y)` đã thay đổi object `ridge`, đặc biệt thêm các attribute như `coef_` vốn chưa tồn tại trước đó.

Estimating Test Error of Ridge Regression

Chọn trước một value λ cho ridge regression là khó, nếu không nói là impossible. Ta muốn dùng validation method hoặc cross-validation để select tuning parameter. Không bất ngờ khi `Pipeline()` có thể được dùng trong `skm.cross_validate()` với validation method (tức `validation`) hoặc k -fold cross-validation. Ta fix random state của splitter để result reproducible.

```
In [30]: validation = skm.ShuffleSplit(n_splits=1,
        test_size=0.5,
        random_state=0)

        ridge.alpha = 0.01
        results = skm.cross_validate(ridge,
        X,
        Y,
        scoring='neg_mean_squared_error',
        cv=validation)

        -results['test_score']
```

```
Out[30]: array([134214.0])
```

Test MSE là $1.342e+05$. Nếu chỉ fit model có intercept, ta sẽ predict mỗi test observation bằng mean của training observations. Có thể lấy cùng result bằng ridge regression với λ rất lớn. Lưu ý $1e10$ nghĩa là 10^{10} .

```
In [31]: ridge.alpha = 1e10
        results = skm.cross_validate(ridge,
        X,
        Y,
        scoring='neg_mean_squared_error',
        cv=validation)

        -results['test_score']
```

```
Out[31]: array([231788.32])
```

Rõ ràng chọn $\lambda = 0.01$ là arbitrary, nên ta dùng cross-validation hoặc validation-set approach để chọn tuning parameter λ . Object `GridSearchCV()` cho exhaustive grid search để chọn parameter như vậy. Trước hết ta dùng validation-set method để chọn λ .

GridSearchCV()

```
In [32]: param_grid = {'ridge__alpha': lambdas}
grid = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=validation,
                        scoring='neg_mean_squared_error')

grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

```
Out[32]: Pipeline(steps=[('scaler', StandardScaler()),
                        ('ridge', ElasticNet(alpha=0.005899, l1_ratio=0))])
```

Alternative, có thể dùng 5-fold cross-validation.

```
In [33]: grid = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=kfold,
                        scoring='neg_mean_squared_error')

grid.fit(X, Y)
grid.best_params_['ridge__alpha']
grid.best_estimator_
```

Nhớ rằng ta setup object `kfold` cho 5-fold cross-validation ở page 271. Bây giờ plot cross-validated MSE như function của $-\log(\lambda)$, trong đó shrinkage giảm từ left sang right.

```
In [34]: ridge_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
            -grid.cv_results_['mean_test_score'],
            yerr=grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylim([50000, 250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated MSE', fontsize=20);
```

Có thể cross-validate các metric khác nhau để chọn parameter. Default metric cho `skl.ElasticNet()` là test R^2 . Ta compare R^2 với MSE cho cross-validation ở đây.

```
In [35]: grid_r2 = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=kfold)

grid_r2.fit(X, Y)
```

Cuối cùng plot result cho cross-validated R^2 .

```
In [36]: r2_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
            grid_r2.cv_results_['mean_test_score'],
            yerr=grid_r2.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated $R^2$', fontsize=20);
```

Fast Cross-Validation for Solution Paths

Ridge, lasso và elastic net có thể được fit hiệu quả dọc sequence of λ values, tạo nên *solution path* hay *regularization path*. Vì vậy có specialized code để fit các path này và chọn suitable λ bằng cross-validation. Dù dùng identical splits, result sẽ không agree hoàn toàn với grid trên vì standard-

ization của mỗi feature trong `grid` được thực hiện riêng trên từng fold, còn trong `pipeCV` bên dưới chỉ thực hiện một lần. Tuy nhiên result tương tự vì normalization khá stable giữa các fold.

```
In [37]: ridgeCV = skl.ElasticNetCV(alphas=lambdas,
                                l1_ratio=0,
                                cv=kfold)
pipeCV = Pipeline(steps=[('scaler', scaler),
                          ('ridge', ridgeCV)])
pipeCV.fit(X, Y)
```

Ta lại tạo plot cross-validation error để thấy nó tương tự khi dùng `skm.GridSearchCV`.

```
In [38]: tuned_ridge = pipeCV.named_steps['ridge']
ridgeCV_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(lambdas),
            tuned_ridge.mse_path_.mean(1),
            yerr=tuned_ridge.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_ridge.alpha_), c='k', ls='--')
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated MSE', fontsize=20);
```

Ta thấy value λ cho smallest cross-validation error là $1.19e-02$, có ở `tuned_ridge.alpha_`. Test MSE tương ứng là bao nhiêu?

```
In [39]: np.min(tuned_ridge.mse_path_.mean(1))
```

```
Out[39]: 115526.71
```

Đây là improvement thêm so với test MSE khi dùng $\lambda = 4$. Cuối cùng `tuned_ridge.coef_` chứa coefficients fit trên entire dataset tại value λ này.

```
In [40]: tuned_ridge.coef_
```

```
Out[40]: array([-222.80877051, 238.77246614,  3.21103754, -2.93050845,
                3.64888723, 108.90953869, -50.81896152, -105.15731984,
                122.00714801,  57.1859509 , 210.35170348, 118.05683748,
               -150.21959435,  30.36634231, -61.62459095,  77.73832472,
                40.07350744, -25.02151514, -13.68429544])
```

Đúng như expected, không coefficient nào bằng zero—ridge regression không perform variable selection.

Evaluating Test Error of Cross-Validated Ridge

Chọn λ bằng cross-validation tạo ra một single regression estimator, tương tự fit linear regression model trong Chapter 3. Vì vậy việc estimate test error của nó là reasonable. Nhưng cross-validation đã “touch” toàn bộ data khi chọn λ , nên không còn data độc lập để estimate test error. Một compromise là split data ban đầu thành hai disjoint sets: training và test. Sau đó fit cross-validation-tuned ridge regression trên training set và evaluate trên test set. Có thể gọi đây là cross-validation nested trong validation-set approach. Không có lý do a priori để chia nửa data cho mỗi set; bên dưới dùng 75% training và 25% test, estimator là ridge regression tuned bằng 5-fold cross-validation.

```
In [41]: outer_valid = skm.ShuffleSplit(n_splits=1,
                                     test_size=0.25,
                                     random_state=1)

inner_cv = skm.KFold(n_splits=5,
                    shuffle=True,
                    random_state=2)

ridgeCV = skl.ElasticNetCV(alphas=lambdas,
                          l1_ratio=0,
                          cv=inner_cv)

pipeCV = Pipeline(steps=[('scaler', scaler),
                          ('ridge', ridgeCV)]);
```

```
In [42]: results = skm.cross_validate(pipeCV,
                                     X,
                                     Y,
                                     cv=outer_valid,
                                     scoring='neg_mean_squared_error')

- results['test_score']
```

```
Out[42]: array([132393.84])
```

The Lasso

Ta đã thấy ridge regression với lựa chọn λ hợp lý có thể outperform least squares và null model trên **Hitters**. Bây giờ hỏi liệu lasso có thể tạo model accurate hơn hoặc interpretable hơn ridge regression hay không. Để fit lasso, ta lại dùng `ElasticNetCV()`, nhưng lần này argument `l1_ratio=1`. Ngoài thay đổi đó, procedure giống ridge.

```
In [43]: lassoCV = skl.ElasticNetCV(n_alphas=100,
                                   l1_ratio=1,
                                   cv=kfold)

pipeCV = Pipeline(steps=[('scaler', scaler),
                          ('lasso', lassoCV)])

pipeCV.fit(X, Y)
tuned_lasso = pipeCV.named_steps['lasso']
tuned_lasso.alpha_
```

```
Out[43]: 3.147
```

```
In [44]: lambdas , soln_array = skl.Lasso.path(Xs,
                                              Y,
                                              l1_ratio=1,
                                              n_alphas=100)[:2]

soln_path = pd.DataFrame(soln_array.T,
                        columns=D.columns,
                        index=-np.log(lambdas))
```

Từ coefficient plot của standardized coefficients có thể thấy tùy tuning parameter, một số coefficients sẽ exactly zero.

```
In [45]: path_fig, ax = subplots(figsize=(8,8))
soln_path.plot(ax=ax, legend=False)
ax.legend(loc='upper left')
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Standardized coefficients', fontsize=20);
```

Smallest cross-validated error thấp hơn test-set MSE của null model và

least squares, và rất gần test MSE 115526.71 của ridge regression (page 278) với λ chọn bằng cross-validation.

```
In [46]: np.min(tuned_lasso.mse_path_.mean(1))
```

```
Out[46]: 114690.73
```

Ta lại tạo plot cross-validation error.

```
In [47]: lassoCV_fig, ax = subplots(figsize=(8,8))
ax.errorbar(-np.log(tuned_lasso.alphas_),
            tuned_lasso.mse_path_.mean(1),
            yerr=tuned_lasso.mse_path_.std(1) / np.sqrt(K))
ax.axvline(-np.log(tuned_lasso.alpha_), c='k', ls='--')
ax.set_ylim([50000,250000])
ax.set_xlabel('$-\log(\lambda)$', fontsize=20)
ax.set_ylabel('Cross-validated MSE', fontsize=20);
```

Tuy nhiên lasso có advantage lớn so với ridge: resulting coefficient estimates sparse. Ở đây 6 trong 19 coefficient estimates exactly zero, nên lasso model với λ chọn bằng cross-validation chỉ chứa 13 variables.

```
In [48]: tuned_lasso.coef_
```

```
Out[48]: array([-210.01008773, 243.4550306 ,  0.          ,  0.          ,
                0.          , 97.69397357, -41.52283116, -0.          ,
                0.          , 39.62298193, 205.75273856, 124.55456561,
               -126.29986768, 15.70262427, -59.50157967, 75.24590036,
                21.62698014, -12.04423675, -0.          ])
```

Giống ridge regression, ta có thể evaluate test error của cross-validated lasso bằng cách split test/training trước rồi chạy cross-validation internally trên training set. Việc này được để lại như exercise.

6.5.3 PCR and PLS Regression

Principal Components Regression

Principal components regression (PCR) có thể được thực hiện bằng PCA() từ module `sklearn.decomposition`. Ta áp dụng PCR cho `Hitters` để predict `Salary`; nhớ bảo đảm missing values đã được remove như Section 6.5.1. Ở đây dùng `LinearRegression()` để fit regression model. Function này fit intercept mặc định, khác `OLS()` ở Section 6.5.1.

```
In [49]: pca = PCA(n_components=2)
linreg = skl.LinearRegression()
pipe = Pipeline([('pca', pca),
                 ('linreg', linreg)])
pipe.fit(X, Y)
pipe.named_steps['linreg'].coef_
```

PCA()
LinearRegression()

```
Out[49]: array([0.09846131, 0.4758765 ])
```

Khi perform PCA, result phụ thuộc việc data có được standardize hay không. Như các example trước, có thể làm bằng cách thêm một step vào pipeline.

```
In [50]: pipe = Pipeline([('scaler', scaler),
                          ('pca', pca),
                          ('linreg', linreg)])
pipe.fit(X, Y)
pipe.named_steps['linreg'].coef_
```

```
Out[50]: array([106.36859204, -21.60350456])
```

Ta có thể dùng CV để chọn number of components bằng `skm.GridSearchCV`; ở đây vary parameter `n_components`.

```
In [51]: param_grid = {'pca__n_components': range(1, 20)}
grid = skm.GridSearchCV(pipe,
                        param_grid,
                        cv=kfold,
                        scoring='neg_mean_squared_error')
grid.fit(X, Y)
```

Plot result giống các method trước.

```
In [52]: pcr_fig, ax = subplots(figsize=(8,8))
n_comp = param_grid['pca__n_components']
ax.errorbar(n_comp,
            -grid.cv_results_['mean_test_score'],
            grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylabel('Cross-validated MSE', fontsize=20)
ax.set_xlabel('# principal components', fontsize=20)
ax.set_xticks(n_comp[:2])
ax.set_ylim([50000, 250000]);
```

Smallest cross-validation error xảy ra khi dùng 17 components. Tuy nhiên plot cũng cho thấy cross-validation error gần như same khi chỉ có one component trong model. Điều này gợi ý model với rất ít components có thể đã đủ.

CV score được cung cấp cho mọi possible number of components từ 1 đến 19. Method `PCA()` phần này nếu cố fit intercept-only với `n_components=0`, nên ta compute thêm MSE cho null model trên chính các split này.

```
In [53]: Xn = np.zeros((X.shape[0], 1))
cv_null = skm.cross_validate(linreg,
                             Xn,
                             Y,
                             cv=kfold,
                             scoring='neg_mean_squared_error')
-cv_null['test_score'].mean()
```

```
Out[53]: 204139.31
```

Attribute `explained_variance_ratio_` của `PCA` object cho percentage of variance explained trong predictors và response khi dùng different numbers of components. Concept này được bàn kỹ hơn ở Section 12.2.

```
In [54]: pipe.named_steps['pca'].explained_variance_ratio_
```

```
Out[54]: array([0.3831424 , 0.21841076])
```

Có thể hiểu đây là lượng information về predictors được capture bởi M principal components. Ví dụ $M = 1$ chỉ capture 38.31% variance, còn $M = 2$ capture thêm 21.84%, tổng 60.15%. Đến $M = 6$ tăng lên 88.63%. Sau đó increment tiếp tục giảm dần, cho tới $M = p = 19$ thì capture đủ 100% variance.

Partial Least Squares

Partial least squares (PLS) được implement bằng function `PLSRegression()`.

PLSRegression()

```
In [55]: pls = PLSRegression(n_components=2,
                           scale=True)
pls.fit(X, Y)
```

Giống PCR, ta muốn dùng CV để chọn number of components.

```
In [56]: param_grid = {'n_components':range(1, 20)}
grid = skm.GridSearchCV(pls,
                        param_grid,
                        cv=kfold,
                        scoring='neg_mean_squared_error')
grid.fit(X, Y)
```

Như các method khác, ta plot MSE.

```
In [57]: pls_fig, ax = subplots(figsize=(8,8))
n_comp = param_grid['n_components']
ax.errorbar(n_comp,
            -grid.cv_results_['mean_test_score'],
            grid.cv_results_['std_test_score'] / np.sqrt(K))
ax.set_ylabel('Cross-validated MSE', fontsize=20)
ax.set_xlabel('# principal components', fontsize=20)
ax.set_xticks(n_comp[:2])
ax.set_ylim([50000,250000]);
```

CV error được minimize tại 12, dù difference với number nhỏ hơn nhiều như 2 hoặc 3 components là rất ít.

6.6 Exercises

Conceptual

1. Ta thực hiện best subset, forward stepwise và backward stepwise selection trên cùng một dataset. Với mỗi approach, ta thu được $p + 1$ models chứa $0, 1, 2, \dots, p$ predictors. Hãy giải thích câu trả lời:
 - (a) Trong ba model có k predictors, model nào có training RSS nhỏ nhất?
 - (b) Trong ba model có k predictors, model nào có test RSS nhỏ nhất?
 - (c) True or False:
 - i. Predictors trong k -variable model do forward stepwise xác định là subset của predictors trong $(k + 1)$ -variable model do forward stepwise selection xác định.
 - ii. Predictors trong k -variable model do backward stepwise xác định là subset của predictors trong $(k + 1)$ -variable model

- do backward stepwise selection xác định.
- iii. Predictors trong k -variable model do backward stepwise xác định là subset của predictors trong $(k + 1)$ -variable model do forward stepwise selection xác định.
 - iv. Predictors trong k -variable model do forward stepwise xác định là subset của predictors trong $(k + 1)$ -variable model do backward stepwise selection xác định.
 - v. Predictors trong k -variable model do best subset xác định là subset của predictors trong $(k + 1)$ -variable model do best subset selection xác định.
2. Với các phần (a)–(c), hãy chỉ ra lựa chọn nào trong i.–iv. là đúng và giải thích.
- (a) Lasso, so với least squares, là:
 - i. More flexible và do đó prediction accuracy sẽ tốt hơn khi increase in bias nhỏ hơn decrease in variance.
 - ii. More flexible và do đó prediction accuracy sẽ tốt hơn khi increase in variance nhỏ hơn decrease in bias.
 - iii. Less flexible và do đó prediction accuracy sẽ tốt hơn khi increase in bias nhỏ hơn decrease in variance.
 - iv. Less flexible và do đó prediction accuracy sẽ tốt hơn khi increase in variance nhỏ hơn decrease in bias.
 - (b) Lặp lại (a) cho ridge regression relative to least squares.
 - (c) Lặp lại (a) cho non-linear methods relative to least squares.
3. Giả sử ta estimate regression coefficients trong linear regression model bằng cách minimize

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2$$

subject to

$$\sum_{j=1}^p |\beta_j| \leq s$$

cho một value s cụ thể. Với (a)–(e), chọn đáp án đúng trong i.–v. và giải thích.

- (a) Khi tăng s từ 0, training RSS sẽ:
 - i. Ban đầu tăng rồi cuối cùng giảm theo inverted U shape.
 - ii. Ban đầu giảm rồi cuối cùng tăng theo U shape.
 - iii. Steadily increase.
 - iv. Steadily decrease.
 - v. Remain constant.
- (b) Lặp lại (a) cho test RSS.
- (c) Lặp lại (a) cho variance.
- (d) Lặp lại (a) cho (squared) bias.

(e) Lặp lại (a) cho irreducible error.

4. Giả sử ta estimate regression coefficients trong linear regression model bằng cách minimize

$$\sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2 + \lambda \sum_{j=1}^p \beta_j^2$$

cho một value λ cụ thể. Với (a)–(e), chọn đáp án đúng trong i.–v. và giải thích.

(a) Khi tăng λ từ 0, training RSS sẽ:

- i. Ban đầu tăng rồi cuối cùng giảm theo inverted U shape.
- ii. Ban đầu giảm rồi cuối cùng tăng theo U shape.
- iii. Steadily increase.
- iv. Steadily decrease.
- v. Remain constant.

(b) Lặp lại (a) cho test RSS.

(c) Lặp lại (a) cho variance.

(d) Lặp lại (a) cho (squared) bias.

(e) Lặp lại (a) cho irreducible error.



5. Ridge regression được biết là thường cho similar coefficient values đối với correlated variables, còn lasso có thể cho coefficient values khá different đối với correlated variables. Ta khảo sát property này trong một setting rất đơn giản.

Giả sử $n = 2$, $p = 2$, $x_{11} = x_{12}$, $x_{21} = x_{22}$. Ngoài ra giả sử $y_1 + y_2 = 0$, $x_{11} + x_{21} = 0$ và $x_{12} + x_{22} = 0$, nên estimate intercept trong least squares, ridge regression hoặc lasso model bằng zero: $\hat{\beta}_0 = 0$.

(a) Viết ridge regression optimization problem trong setting này.

(b) Lập luận rằng trong setting này, ridge coefficient estimates thỏa $\hat{\beta}_1 = \hat{\beta}_2$.

(c) Viết lasso optimization problem trong setting này.

(d) Lập luận rằng trong setting này, lasso coefficients $\hat{\beta}_1$ và $\hat{\beta}_2$ không unique—tức có nhiều possible solution cho optimization problem trong (c). Mô tả các solution này.

6. Ta sẽ khảo sát thêm (6.12) và (6.13).

(a) Xét (6.12) với $p = 1$. Với một lựa chọn y_1 và $\lambda > 0$, plot (6.12) như function của β_1 . Plot phải confirm rằng (6.12) được giải bởi (6.14).

(b) Xét (6.13) với $p = 1$. Với một lựa chọn y_1 và $\lambda > 0$, plot (6.13) như function của β_1 . Plot phải confirm rằng (6.13) được giải bởi (6.15).



7. Ta sẽ derive Bayesian connection với lasso và ridge regression đã bàn trong Section 6.2.2.

- (a) Giả sử $y_i = \beta_0 + \sum_{j=1}^p x_{ij}\beta_j + \varepsilon_i$, trong đó $\varepsilon_1, \dots, \varepsilon_n$ independent và identically distributed theo $N(0, \sigma^2)$. Viết likelihood cho data.
- (b) Assume prior sau cho β : $\beta_1, \dots, \beta_p$ independent và identically distributed theo double-exponential distribution với mean 0 và common scale parameter b , tức

$$p(\beta) = \frac{1}{2b} \exp(-|\beta|/b).$$

Viết posterior cho β trong setting này.

- (c) Lập luận rằng lasso estimate là mode của β dưới posterior distribution này.
- (d) Bây giờ assume prior sau cho β : $\beta_1, \dots, \beta_p$ independent và identically distributed theo normal distribution với mean zero và variance c . Viết posterior cho β trong setting này.
- (e) Lập luận rằng ridge regression estimate vừa là mode vừa là mean của β dưới posterior distribution này.

Applied

8. Trong exercise này, ta generate simulated data rồi dùng data đó để perform forward và backward stepwise selection.

- (a) Tạo random number generator và dùng method `normal()` để generate predictor X length $n = 100$, cùng noise vector ε length $n = 100$.
- (b) Generate response vector Y length $n = 100$ theo model

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \beta_3 X^3 + \varepsilon,$$

trong đó $\beta_0, \beta_1, \beta_2, \beta_3$ là constants do bạn chọn.

- (c) Dùng forward stepwise selection để select model chứa predictors $X, X^2, \dots, X^{10}$. Model thu được theo C_p là gì? Report coefficients của model đó.
- (d) Lặp lại (c) bằng backwards stepwise selection. Câu trả lời so với (c) thế nào?
- (e) Fit lasso model cho simulated data, vẫn dùng $X, X^2, \dots, X^{10}$ làm predictors. Dùng cross-validation để select optimal λ . Tạo plot cross-validation error theo λ . Report resulting coefficient estimates và discuss results.
- (f) Generate response vector Y theo model

$$Y = \beta_0 + \beta_7 X^7 + \varepsilon,$$

rồi perform forward stepwise selection và lasso. Discuss results.

9. Trong exercise này, ta predict number of applications received bằng các variables khác trong dataset `College`.

- (a) Split dataset thành training set và test set.
- (b) Fit linear model bằng least squares trên training set và report test error.

- (c) Fit ridge regression model trên training set, với λ chọn bằng cross-validation. Report test error.
 - (d) Fit lasso model trên training set, với λ chọn bằng cross-validation. Report test error cùng number of non-zero coefficient estimates.
 - (e) Fit PCR model trên training set, với M chọn bằng cross-validation. Report test error cùng value M được cross-validation chọn.
 - (f) Fit PLS model trên training set, với M chọn bằng cross-validation. Report test error cùng value M được cross-validation chọn.
 - (g) Comment các result thu được. Ta có thể predict number of college applications received chính xác đến mức nào? Test error từ năm approach này có khác biệt nhiều không?
10. Ta đã thấy khi number of features dùng trong model tăng, training error nhất định giảm, nhưng test error có thể không giảm. Ta khảo sát điều này trong simulated dataset.
- (a) Generate dataset có $p = 20$ features, $n = 1,000$ observations, cùng quantitative response vector tạo theo model

$$Y = X\beta + \varepsilon,$$
 trong đó β có một số element exactly zero.
 - (b) Split dataset thành training set gồm 100 observations và test set gồm 900 observations.
 - (c) Perform best subset selection trên training set và plot training-set MSE associated với best model ở mỗi size.
 - (d) Plot test-set MSE associated với best model ở mỗi size.
 - (e) Test-set MSE đạt minimum ở model size nào? Comment result. Nếu minimum xảy ra với intercept-only model hoặc model chứa toàn bộ features, hãy điều chỉnh cách generate data trong (a) cho tới khi có scenario mà test-set MSE minimum ở intermediate model size.
 - (f) Model tại đó test-set MSE minimum so với true model dùng generate data như thế nào? Comment coefficient values.
 - (g) Tạo plot hiển thị $\sqrt{\sum_{j=1}^p (\beta_j - \hat{\beta}_j^r)^2}$ cho một range r , trong đó $\hat{\beta}_j^r$ là j th coefficient estimate của best model chứa r coefficients. Comment điều quan sát được. So sánh với test-MSE plot từ (d).
11. Ta thử predict per-capita crime rate trong dataset **Boston**.
- (a) Thử một số regression method trong chapter này như best subset selection, lasso, ridge regression và PCR. Present và discuss result cho các approach được xét.
 - (b) Propose một model (hoặc set of models) có vẻ perform tốt trên dataset này và justify answer. Hãy bảo đảm evaluate model performance bằng validation-set error, cross-validation hoặc reasonable alternative khác, thay vì training error.

- (c) Model được chọn có involve toàn bộ features trong dataset không? Vì sao?